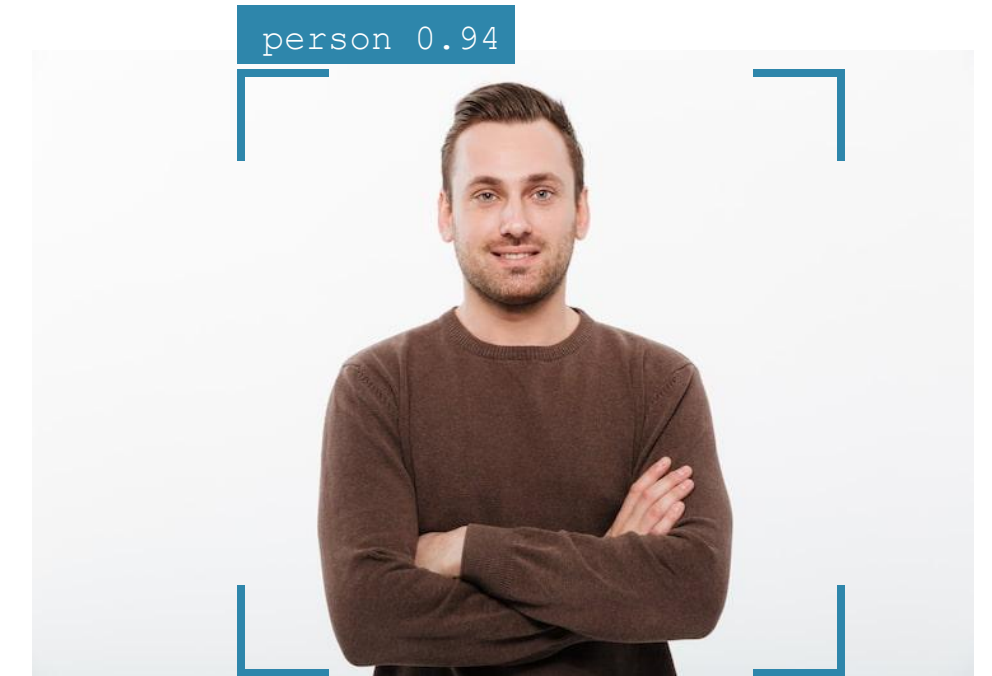


Edge AI & Machine Learning Inference

Best practices for real-time, on-device deployment on Windows — VS Code, Python, YOLO & ONNX Runtime.



What we'll cover

01 Foundations & objectives
What you'll be able to build

02 Introduction to Edge AI
Edge vs. cloud, why it matters

03 Theoretical background
Pipeline & optimization

04 Hardware & software
Choosing the right stack

05 Environment setup
Python, VS Code, virtual envs

06 Model deployment
YOLO, ONNX, benchmarking

07 Advanced applications
ROI, counting, alerting

08 Tracking & analytics
Identity across time

Learning objectives

On completion of this guide and its labs, learners will be able to:

- 01

Explain the principles of Edge AI and articulate the trade-offs of edge vs. cloud deployment.
- 02

Configure a professional Python environment on Windows with VS Code and virtual environments.
- 03

Deploy pre-trained YOLO models for real-time inference using PyTorch and ONNX Runtime.
- 04

Implement a complete webcam detection pipeline: capture, preprocess, infer, post-process, visualize.
- 05

Measure and optimize performance — latency, FPS, mAP — with quantization and runtime tuning.
- 06

Build multi-object tracking and video analytics for production edge deployments.

COMPETENCY TARGET

Independently design, build, deploy, benchmark, and optimize a production-grade Edge AI inference pipeline on a standard Windows workstation — with no cloud dependencies.

SECTION

02

Introduction to Edge AI & ML Inference

What edge inference is, how it differs from the cloud, and why it has become a defining trend in modern computing.

What is Edge AI?

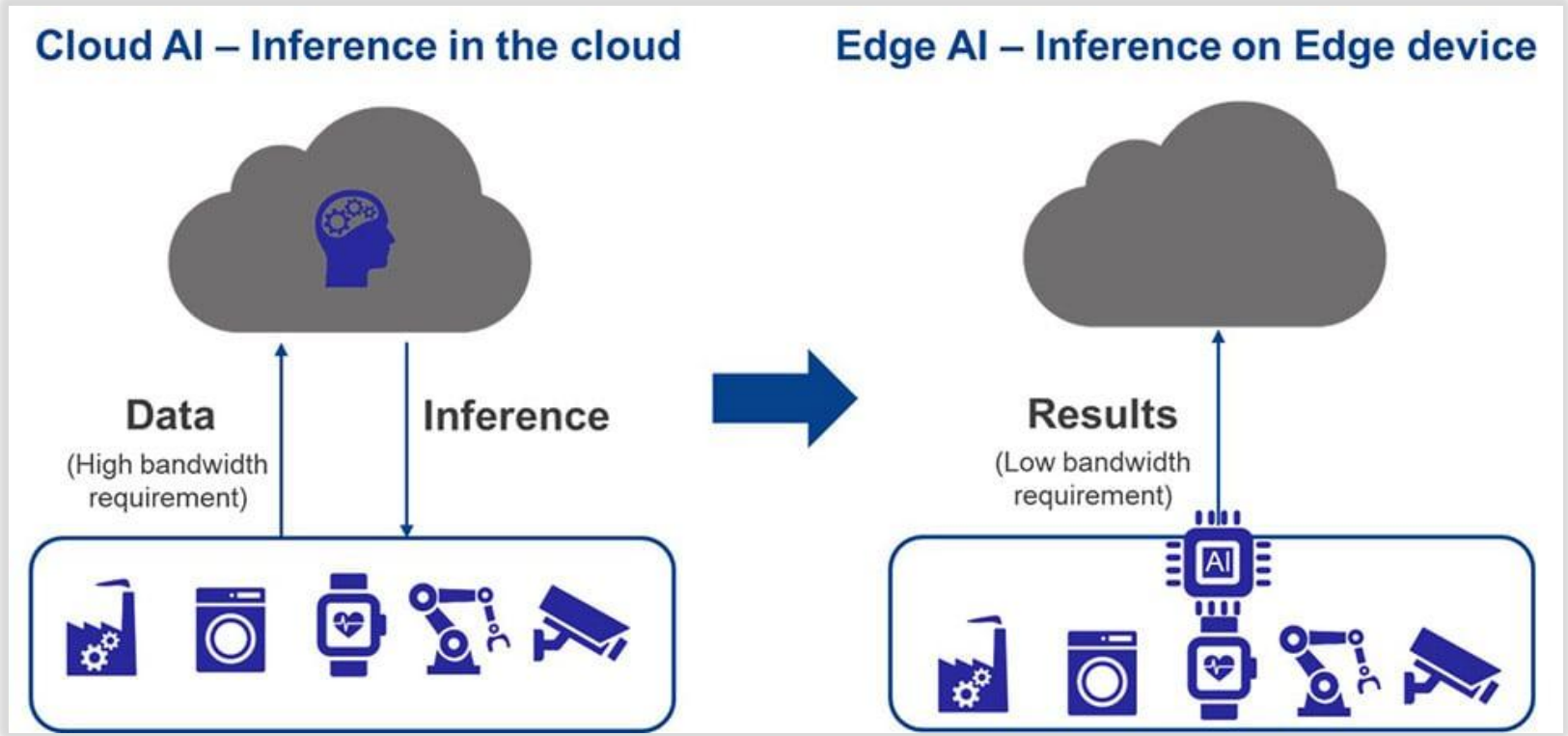
Edge AI runs trained AI algorithms **directly on local hardware** — microcontrollers, single-board computers, smartphones, embedded systems — rather than in centralized cloud data centers, bringing computation physically closer to the source of data.

CLOUD AI

Sensor data is transmitted across the network to remote servers for processing, then the result travels back.

EDGE AI

Data is processed locally at, or near, the point of generation — enabling real-time decisions with no round-trip.



- CRITICAL FOR
- Autonomous vehicles
- Industrial robotics
- Real-time quality inspection
- Medical device monitoring
- Smart Security
- Agriculture

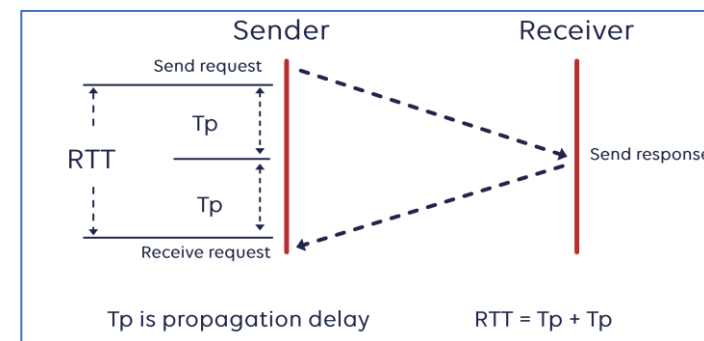
Edge AI vs. Cloud AI

DIMENSION	LOCAL / EDGE AI	CLOUD AI	WHEN LOCAL WINS
Latency	1–50 ms on-device	50–500+ ms round-trip	Real-time, safety-critical
Privacy	Data stays on machine	Data leaves premises	Healthcare, security, IP
Bandwidth	Minimal (metadata)	High (raw upload)	Air-gapped, low-BW sites
Cost at scale	One-time hardware	Recurring API / GPU fees	Continuous inference
Model power	GPU / NPU-dependent	Unlimited (clusters)	Cloud wins for large LLMs
Scalability	Per-machine deploy	Elastic, on-demand	Cloud wins for burst load

Why Edge AI matters

01 Latency reduction

Eliminates the network round-trip — milliseconds instead of hundreds.
Decisive for safety-critical control.



02 Privacy & data sovereignty

Sensitive data never leaves the device — essential under HIPAA, GDPR, and CCPA.



03 Bandwidth & offline

Transmits only lightweight metadata, and keeps working with no internet connection at all.

04 Cost efficiency

No recurring per-request fees — existing hardware has near-zero marginal compute cost.

The decisive difference is response time

ON-DEVICE

5–50ms

Local inference on a Windows PC — no network in the loop.

CLOUD ROUND-TRIP

50–500ms

Network call to a remote server and back, varying with connectivity.

SAFETY-CRITICAL For autonomous vehicles, robotic surgery, and emergency shutoffs, even **100 ms** of extra latency can be the difference between a safe stop and a collision.

Privacy & data sovereignty

Processing on a local machine keeps sensitive information within the user's physical control. With Edge AI, raw video frames, biometric data, and patient records **never traverse a network.**

- HIPAA
- GDPR
- CCPA

- Medical images & patient records
- Security & surveillance footage
- Proprietary manufacturing data
- Biometric & identity data

Process locally, transmit almost nothing

1.5Gbps

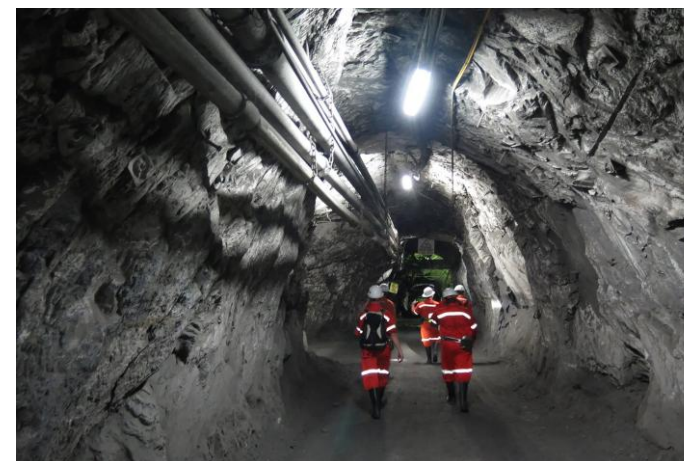
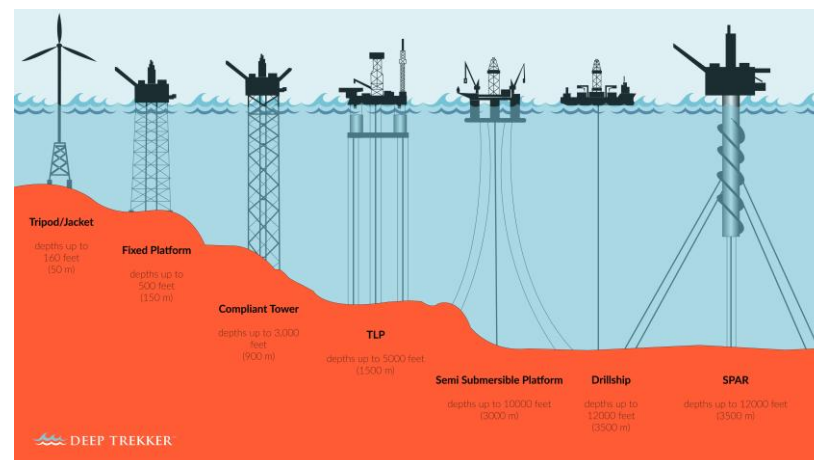
raw data from a single 1080p camera at
30 FPS

- Local inference sends only lightweight **metadata or alerts** , cutting bandwidth by orders of magnitude.
- Systems keep functioning **fully offline** — reliable in connectivity-challenged environments.

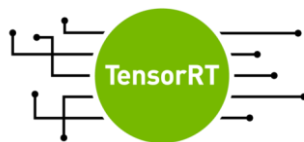
Offshore platforms

Underground mines

Rural agriculture



Common Frameworks & Deployment Tools



Framework	Vendor	Target Platform
TensorFlow Lite	Google	Mobile, embedded, microcontrollers
ONNX Runtime	Microsoft	Cross-platform, multi-framework
TensorRT	NVIDIA	GPU-optimized inference
Core ML	Apple	iOS/macOS native inference
OpenVINO	Intel	Intel CPU/GPU/VPU optimization
MediaPipe	Google	Real-time ML pipelines

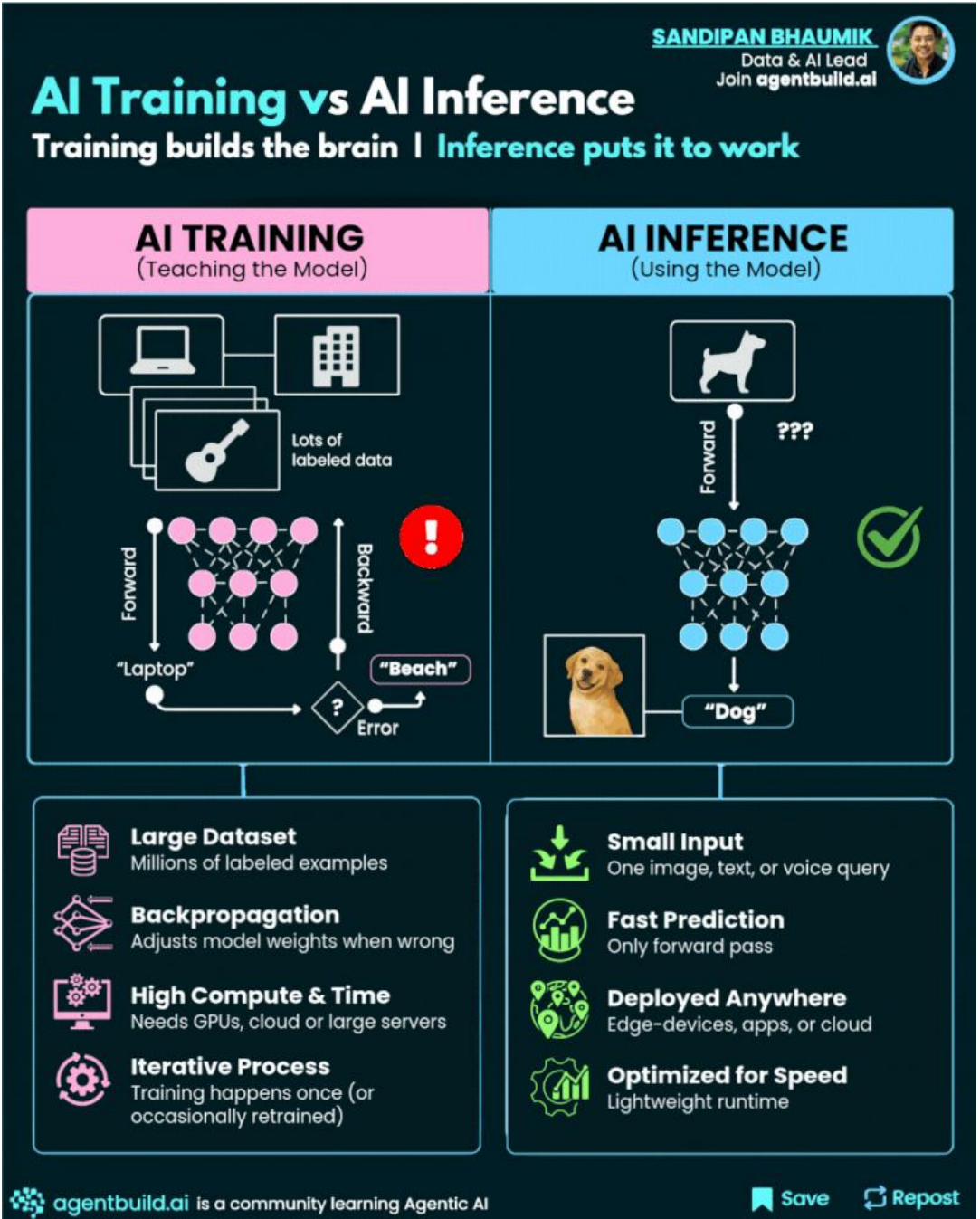
Choose based on target hardware, model format, and deployment constraints.

Inference vs. training

In machine learning, **AI inference** is the act of using a trained AI model to make predictions on new data.

ASPECT	TRAINING	INFERENCE
Definition	Learn parameters from labeled data	Predict on new data with a trained model
Compute	Extremely high — GPU/TPU clusters	Moderate to low — runs on edge devices
Data volume	Thousands to millions of samples	A single sample at a time
Duration	Hours to weeks per run	Milliseconds to seconds per prediction
Optimize for	Accuracy, convergence	Latency, throughput, memory, power
Frequency	Periodic — retrain on new data	Continuous — every frame

This workshop focuses exclusively on inference — deploying pre-trained models for real-time prediction.



Five layers, working together



INTEGRATION

MQTT, gRPC, REST, and hardware interfaces (USB, CSI, GPIO) linking the pipeline to sensors, actuators & dashboards



APPLICATION

Software that orchestrates the pipeline — capture input, invoke the model, process & present results

← Today



MODEL

The trained network, optimized via quantization / pruning / distillation — format must match the runtime

← Today



RUNTIME

The inference engine — ONNX Runtime, TensorRT, TensorFlow Lite, OpenVINO — tuned per hardware family

← Today



HARDWARE

The physical compute — CPU, GPU, NPU, TPU, VPU, FPGA — setting the ceiling for speed & power

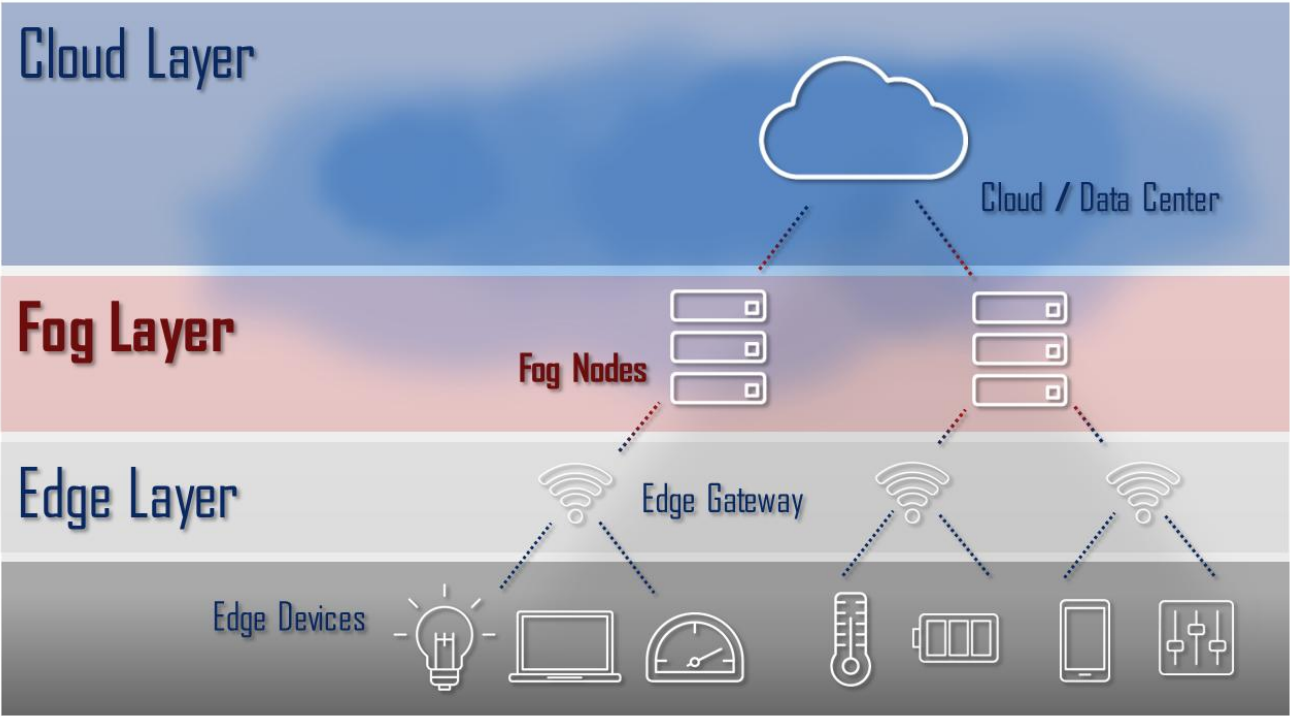
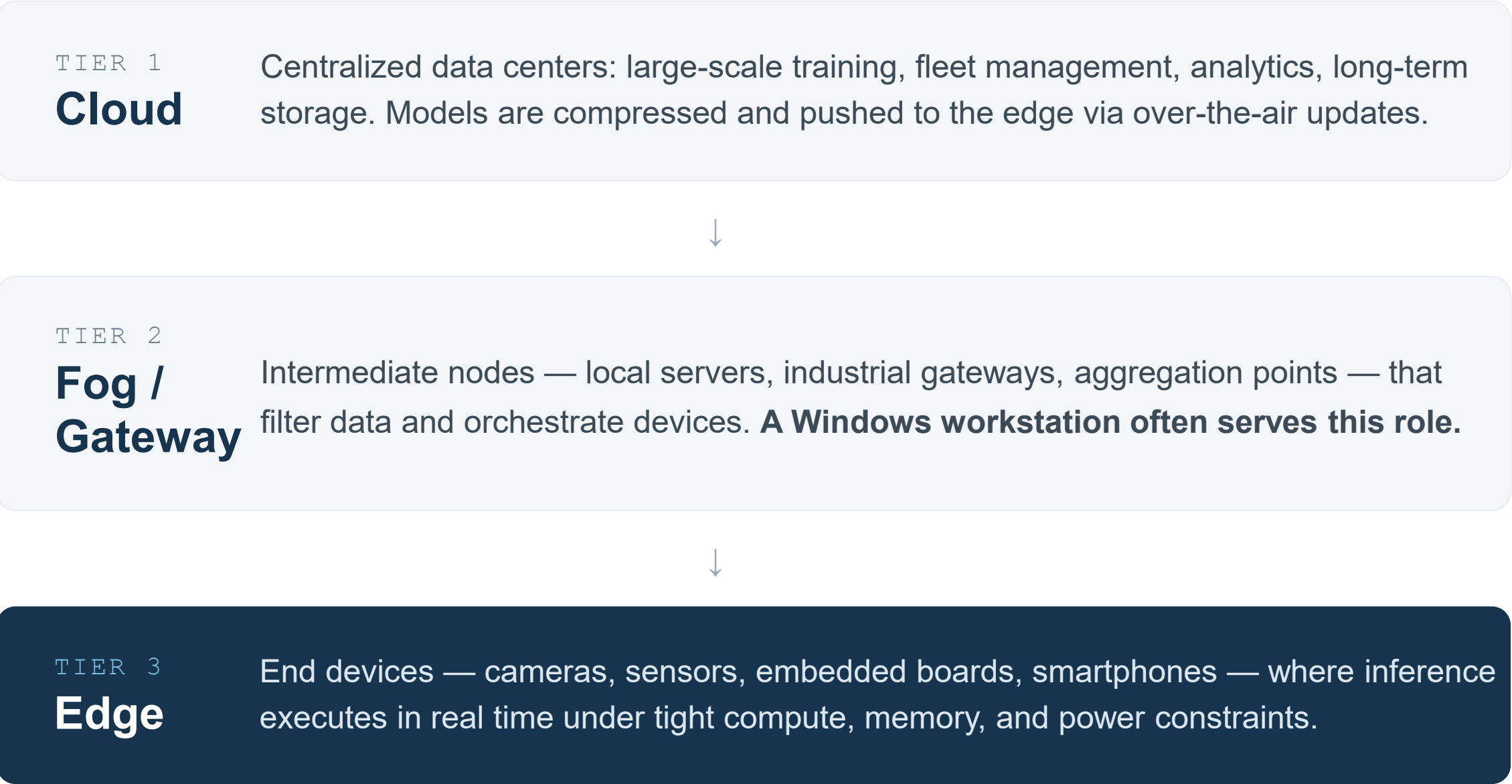
SECTION

03

Theoretical Background

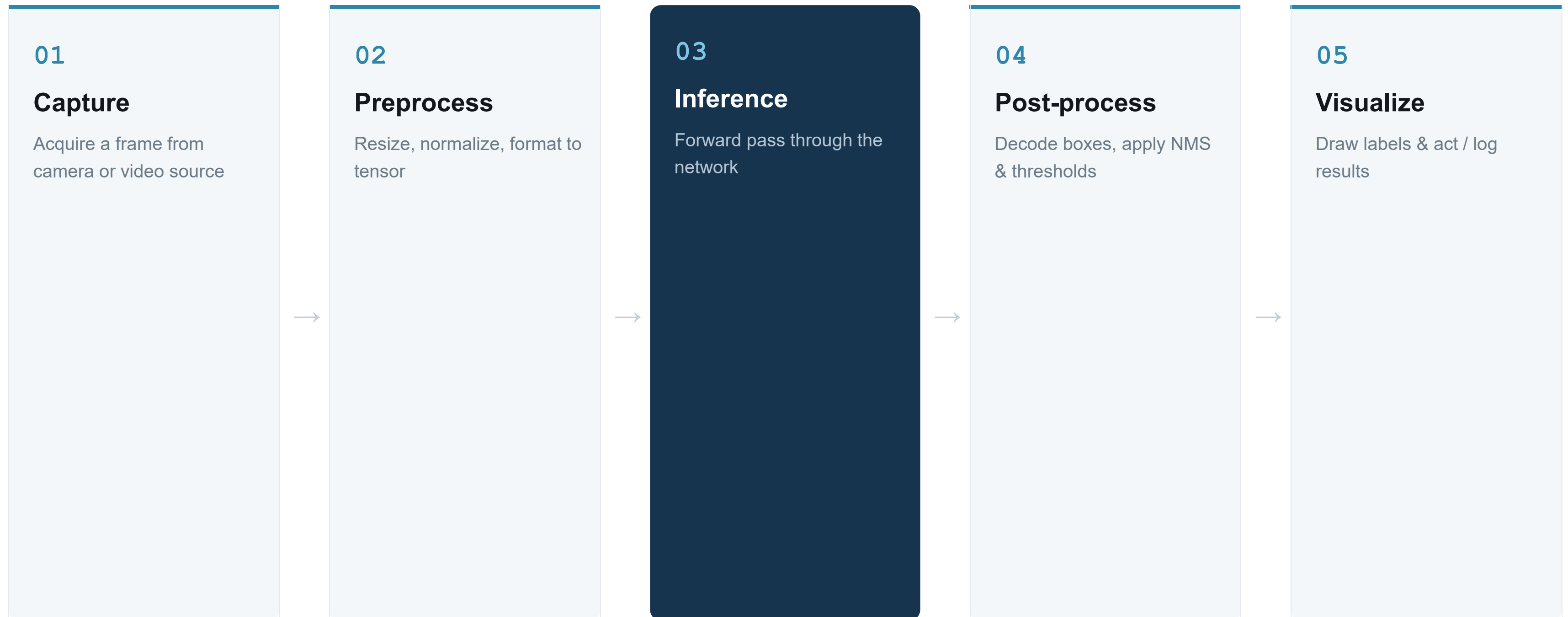
The architecture, the inference pipeline, and the optimization techniques that let trained networks run on constrained hardware.

The three-tier model



Six stages, every frame

The same sequence runs on every device — each stage a place to measure and optimize.



Making models fit the edge

Balancing three competing objectives — **accuracy**, **speed**, and **resource use** — with four families of technique.

01

Quantization | បរិមាណនីយកម្ម

Lower numerical precision — FP16, INT8

FP16 and **INT8** are low-precision data types commonly used in deep learning and AI to reduce memory usage and speed up processing. **FP16** is a 16-bit floating-point format that retains fractional values for good accuracy. **INT8** is an 8-bit integer format used for quantization, which shrinks models by dropping fractional precision to maximize hardware efficiency

02

Pruning | ការកាត់ចេញ

Remove redundant weights & filters

03

Distillation | ការចម្រាញ់

Small student mimics a large teacher

04

Architecture | ស្ថាបត្យកម្ម

Pick an inherently efficient model

Trading precision for speed

Reduce weights and activations from 32-bit float (FP32) to lower precision — shrinking the model and accelerating compute.

FP16

Half-precision float

~50%

smaller, <1% accuracy loss

Default for GPU — NVIDIA Tensor Cores, Apple Neural Engine.

INT8

8-bit integer

~75%

smaller, needs calibration

Maximum speed-up on NPUs & CPUs; validate carefully.

DYNAMIC

Runtime activations

Easy

no calibration data

Simplest to apply; smaller gains than static INT8.

For most YOLO detection tasks, **FP16 has negligible accuracy impact** — make it your default for any GPU deployment. Always benchmark accuracy after quantizing.

Choosing an efficient model

Often the single most impactful optimization — start from an architecture built for constrained hardware.

MobileNetV3

Depthwise-separable convolutions and squeeze-and-excitation blocks — strong accuracy at minimal FLOPs for mobile & embedded CPUs.

YOLO v8 / v11

The de facto standard for real-time edge detection — variants from **Nano (3.2M)** to **Extra-Large (68.2M)** parameters.

ShuffleNet SqueezeNet

Ultra-lightweight for microcontroller-class devices with severe memory limits — under 5 MB model size.

YOLO — You Only Look Once

TWO-STAGE — Faster R-CNN

First propose regions, *then* classify each — accurate but slow.

VS

SINGLE-STAGE — YOLO

One forward pass over the whole image — fast enough for real time.

MODERN YOLO (v8+)

- **Multi-task:** detection, instance segmentation, pose estimation, classification & OBB
- **Size variants:** Nano through Extra-Large — pick your accuracy/speed point
- **Built-in export:** ONNX, TensorRT, CoreML, TFLite, OpenVINO via one `model.export()` call

SECTION

04

Hardware & Software Requirements

What a standard Windows workstation needs — plus the edge platforms and software stack for production deployment.

Minimum vs. recommended

COMPONENT	MINIMUM	RECOMMENDED
Processor	Intel i5 (8th Gen) / Ryzen 5 3600	Intel i7/i9 (12th Gen+) / Ryzen 7 7700+
Memory	8 GB DDR4	16–32 GB DDR4 / DDR5
Graphics	Integrated (CPU-only mode)	NVIDIA RTX 3060+ · CUDA, 6 GB+ VRAM
Storage	20 GB free SSD	50 GB+ NVMe SSD
Webcam	USB 2.0, 720p @ 30 FPS	USB 3.0, 1080p @ 30+ FPS
OS	Windows 10 (64-bit), 1903+	Windows 11 (64-bit), latest

Edge hardware platforms

TOPS (Trillions of Operations Per Second) is the primary metric used to measure the processing power of an NPU (Neural Processing Unit) in AI hardware. [[1](#), [2](#), [3](#)]

AI Performance is commonly evaluated via **Floating Point Operations Per Second (FLOPS)**, which dictate how many calculations a processor can handle.

PLATFORM	PROCESSOR	AI PERF	TYPICAL USE
Jetson Orin Nano	6-core ARM A78AE + Ampere GPU	40 TOPS	Robotics, drones
Jetson Nano (legacy)	Quad ARM A57 + 128-core Maxwell	472 GFLOPS	Education, prototyping
Raspberry Pi 5 + Hailo-8L	Quad ARM A76 + 13 TOPS NPU	13 TOPS	IoT gateways
Google Coral Dev Board	NXP i.MX 8M + Edge TPU	4 TOPS	TFLite inference
Intel NUC + OpenVINO	Intel Core Ultra + integrated NPU	~11 TOPS	Retail analytics

Four pieces to install

Python

3.10 / 3.11 · 64-bit

Primary language for AI/ML. **Avoid 3.12+** — some packages lack compatible builds.

VS Code

latest stable

The IDE — IntelliSense, debugging, integrated terminal, and an extension ecosystem.

Git

latest

Version control for your project and reproducible experiments.

CUDA Toolkit

optional

Only if you have a compatible NVIDIA GPU — unlocks hardware-accelerated inference.

Eight packages, one requirements.txt

Everything the labs need, pinned to known-good minimum versions.

<code>ultralytics</code>	YOLO models & training API
<code>opencv-python</code>	Camera capture & drawing
<code>numpy</code>	Array & tensor math
<code>pandas</code>	Tabular results & logs

<code>torch</code>	Deep-learning backend
<code>onnxruntime</code>	Portable, optimized inference
<code>matplotlib</code>	Benchmark charts
<code>psutil</code>	CPU & memory monitoring

Tooling that pays for itself

Python	IntelliSense, debugging, linting & virtual-environment integration
Pylance	Fast static type checking and rich language support
Jupyter	Interactive notebooks for exploration and quick experiments
Remote — SSH	Develop remotely on a Jetson, Raspberry Pi, or cloud VM
indent-rainbow	Color-coded indentation for Python's whitespace-sensitive syntax

SECTION

05

Environment Setup Procedures

Python, VS Code, and an isolated virtual environment — built carefully so the labs just work.

One checkbox prevents most failures

- 01 Download **Python 3.11.x (64-bit)** from python.org/downloads — not 3.12+
- 02 Run the installer
- 03 Check "**Add Python to PATH**" — the box that matters most
- 04 Verify in a fresh terminal

```
> python --version
Python 3.11.x

> pip --version
pip 24.x
```

⚠ COMMON ERROR

"python is not recognized"
means the PATH box was skipped.
Uninstall completely, restart, and
reinstall with **Add to PATH**
checked.

From editor to Python IDE

- 01 Download from **code.visualstudio.com** and run the installer
- 02 Check "**Add to PATH**" and "**Register Code as an editor**"
- 03 Install the **Python** extension from the marketplace
- 04 Select the interpreter — `Ctrl+Shift+P` → "Python: Select Interpreter"
- 05 Open the integrated terminal `Ctrl+`` and confirm the version

Organize before you code

```
edge-ai-project/  
├─ venv/# virtual environment  
├─ models/# .pt · .onnx · .engine  
├─ src/# source code  
├─ data/# test clips & images  
├─ outputs/# logs & snapshots  
└─ requirements.txt
```

A predictable layout keeps models, code, and results separate — and makes the project easy to share or move to an edge device.

Keep `venv/` out of version control — commit `requirements.txt` instead.

Isolate every project

A mandatory best practice — **never install AI packages into system Python**. A venv keeps dependencies per-project and reproducible.

```
# navigate to the project folder
> cd C:\...\edge-ai-project

# create the environment
> python -m venv venv

# activate it
> venv\Scripts\activate

(venv) > _
```

POWERSHELL FIX

If you see "running scripts is disabled" , run once:

```
Set-ExecutionPolicy
RemoteSigned -Scope CurrentUser
```


One file, one command

```
# requirements.txt
ultralytics>=8.2.0
opencv-python>=4.8.0
numpy>=1.24.0
torch>=2.1.0
onnxruntime>=1.16.0
matplotlib>=3.7.0
psutil>=5.9.0
pandas>=2.0.0
```

```
# install everything
(venv) > pip install -r
requirements.txt
```

HAVE A CUDA GPU?

```
pip uninstall onnxruntime -y
pip install onnxruntime-gpu
```

CUDA (Compute Unified Device Architecture) is NVIDIA's proprietary parallel computing platform. It allows developers to use compatible GPUs for general-purpose processing. By offloading complex calculations from the CPU to the thousands of cores in an NVIDIA GPU, it dramatically accelerates AI, machine learning, and high-performance computing. [[1](#), [2](#), [3](#)]



Four checks before the labs

01 PyTorch & GPU

```
torch.cuda.is_available()
```

02 OpenCV

```
import cv2 · camera access
```

03 Ultralytics / YOLO

```
from ultralytics import YOLO
```

04 ONNX Runtime

```
ort.get_available_providers()
```

✓ Environment verification complete. All systems operational.

SECTION

06

Model Deployment & Configuration

Load YOLO, run live detection, export to ONNX, and benchmark the runtimes — five hands-on labs.

YOLO — You Only Look Once



The industry standard for real-time object detection at the edge — single forward pass through the network.



Single-Stage Detection

Predicts boxes AND class probabilities simultaneously in one pass — no proposal generation bottleneck



Anchor-Free (v8+)

Direct keypoint prediction — better generalization to unusual object shapes and aspect ratios



Multiple Sizes

Nano (3.2M params) to Extra-Large (68.2M params) — choose based on hardware constraints



Multi-Task Support

Detection, segmentation, pose estimation, classification, oriented bounding boxes — one API

Model Comparison

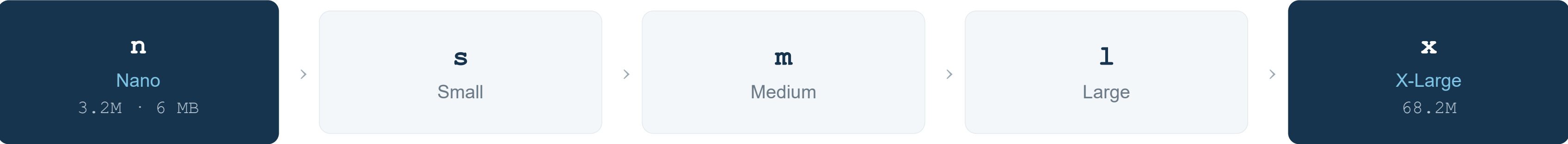
Model	Params	mAP50	CPU ms	
YOLOv8n	3.2M	37.3	~80ms	★
YOLOv8s	11.2M	44.9	~130ms	
YOLOv8m	25.9M	50.2	~240ms	
YOLOv8l	43.7M	52.9	~400ms	
YOLOv8x	68.2M	53.9	~600ms	

★ We use YOLOv8n (Nano) for all lab exercises

One line to a working model

```
from ultralytics import YOLO model = YOLO('yolov8n.pt')
```

Models auto-download on first use and cache locally. Pick a size to trade speed against accuracy:



← faster, smaller

more accurate →

Inspect what the model knows

OBJECTIVE

Load YOLOv8 Nano and print its architecture, parameter count, and supported classes.

STEPS

- 01 Create lab_6_1_model_info.py
- 02 Load the model and read model.names
- 03 Print the total parameter count

```
=====
YOLOv8 NANO - MODEL SUMMARY
=====
Model File      : yolov8n.pt
Training Dataset : MS COCO 2017
Number of Classes : 80
Total Parameters : 3,157,200
Trainable Parameters: 0
=====
SUPPORTED OBJECT CLASSES
-----
[00] person
[01] bicycle
.
.
.
[78] hair drier
[79] toothbrush
=====
```

EXPECTED OUTPUT

```
YOLOv8 NANO - MODEL SUMMARY

Classes : 80 (COCO)
          person, bicycle, car,
          motorcycle, ... toothbrush

Params : ~3,157,200
Task : detection
```

Your first live pipeline

OBJECTIVE

Run real-time object detection on the webcam feed — the exercise that validates the whole pipeline.

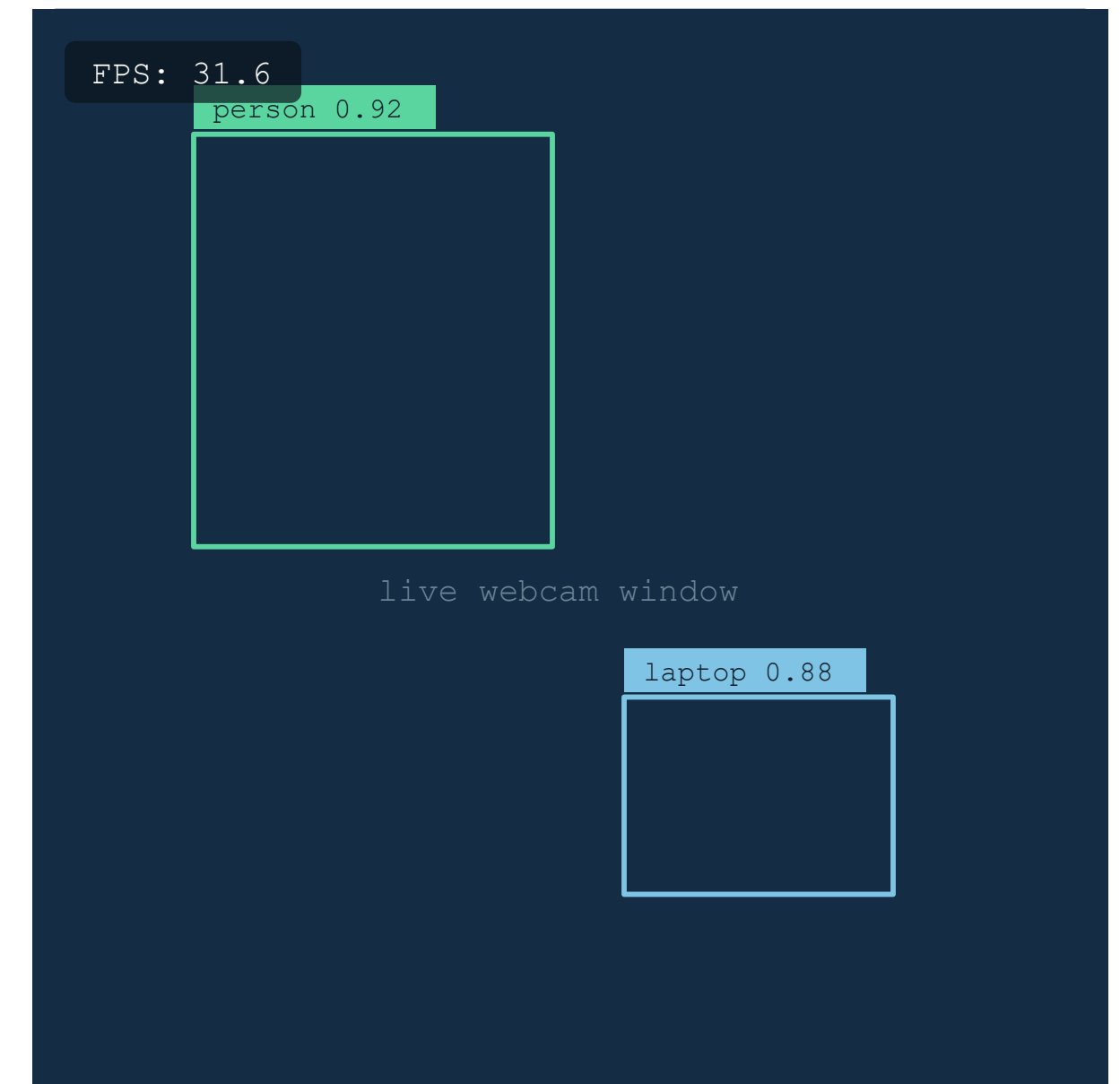
KEY STEPS

- 01 Open the camera — `cv2.VideoCapture(0, cv2.CAP_DSHOW)`
- 02 Per frame: `model(frame) → results[0].plot()`
- 03 Show the annotated frame; press **Q** to quit

Lab Exercise 6.2: Real-Time Webcam Detection

This is the foundational exercise that validates your entire Edge AI pipeline. Create `lab_6_2_webcam_detection.py`:

```
# Lab Exercise 6.2 Practice Real-Time Webcam Detection.py
# — Real-Time Object Detection
```



Black screen? Close Zoom / Teams / OBS and try camera index 1 or 2.

One model, any runtime

Open Neural Network Exchange — an open standard for representing ML models, enabling portability across inference engines and hardware platforms.

Portability

The same model runs across frameworks and toolchains — train once, deploy anywhere.

Hardware acceleration

Execution providers — CUDA, TensorRT, DirectML, OpenVINO, CoreML — use whatever's available.

Smaller footprint

~50 MB for ONNX Runtime vs **~2 GB** for full PyTorch — critical at the edge.

From .pt to portable inference

Lab Exercise 6.3: Export YOLOv8n to ONNX

Export YOLOv8n to ONNX

```
model.export(format='onnx')
```

✓ yolov8n.onnx created in project dir

6.4 · RUN

ONNX Runtime inference

```
# expected
Providers: [CUDA, CPU]

Input : images
        [1, 3, 640, 640]

Output : output0
        [1, 84, 8400]
```

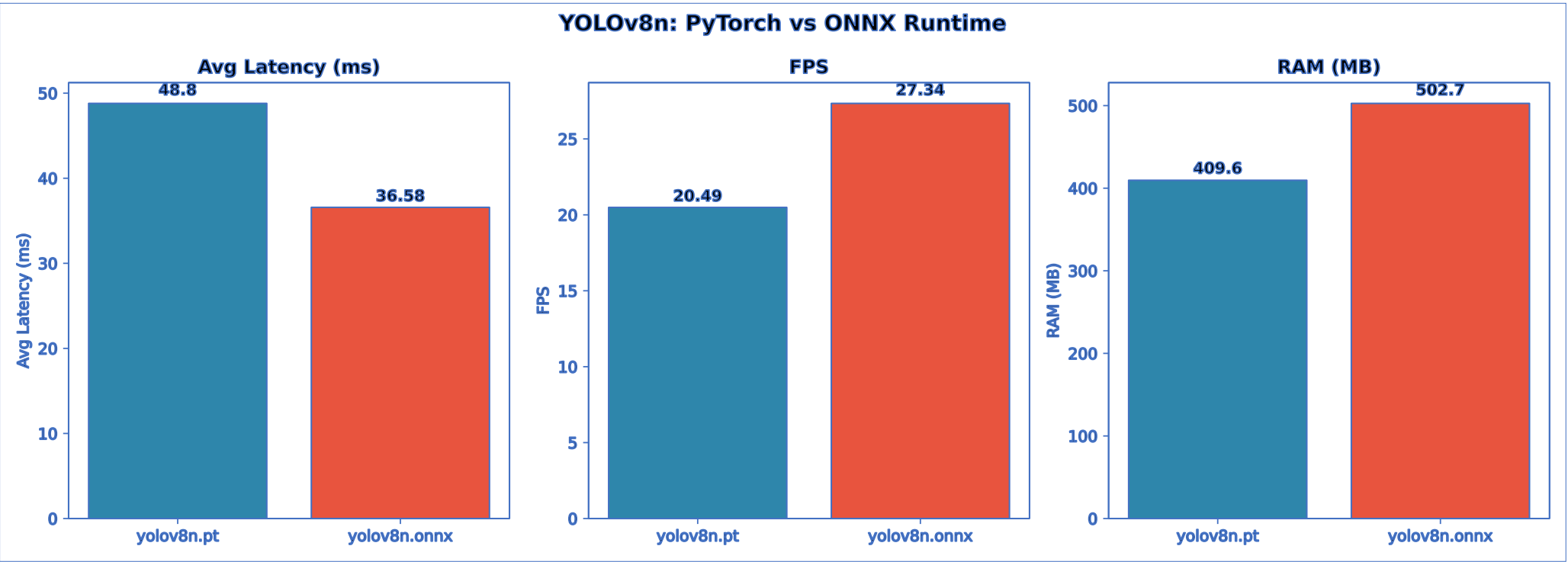
Measure, don't guess

OBJECTIVE

Benchmark both formats under identical conditions and produce a visual comparison of latency and throughput.

WHY WARM-UP MATTERS

Discard the first **10 runs** . They include lazy initialization, memory allocation, and cache population — not steady-state speed.



EXPECTED — RELATIVE FPS

PyTorch (.pt) baseline

ONNX Runtime faster on CPU

illustrative — your numbers depend on hardware

SECTION

07

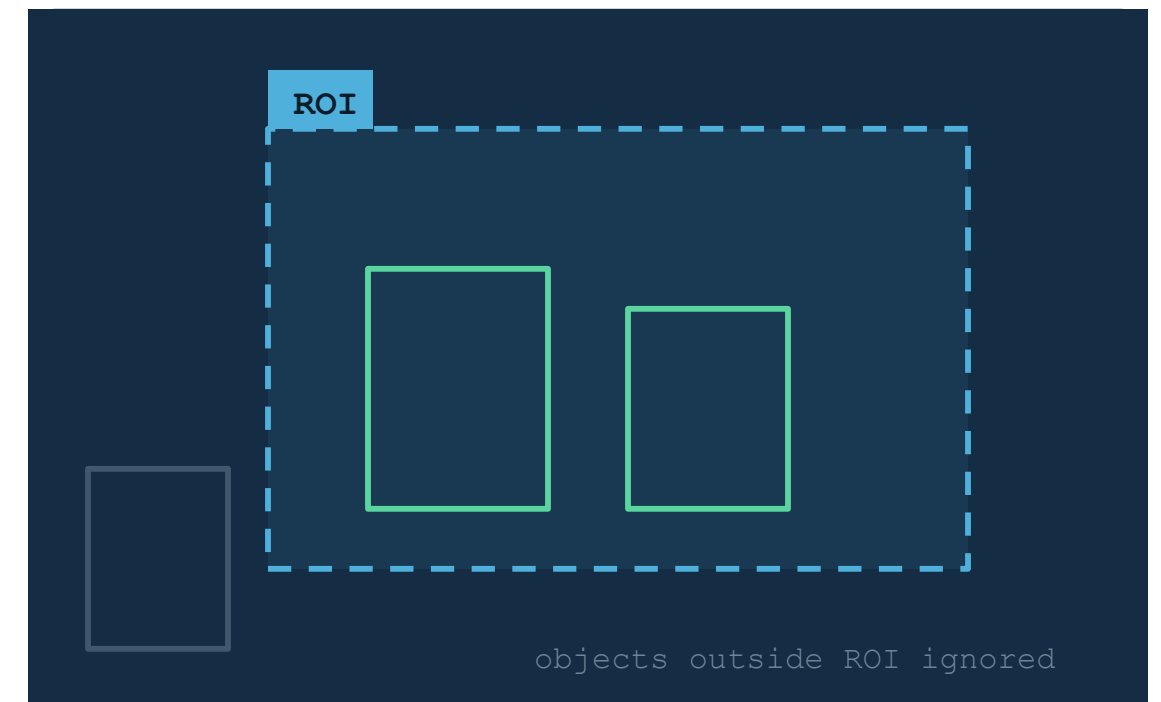
Advanced Real-Time Applications

Region-of-interest filtering, class-specific counting, and alerting — the patterns behind production edge systems.

Watch only what matters

You rarely need to detect across the entire field of view. **ROI filtering** restricts processing to defined areas — **reducing false positives and computational load** at once.

- A warehouse watches only the **loading dock**
- A store monitors the **checkout area**
- A traffic system tracks a **single lane**



Draw a zone, count inside it

OBJECTIVE

Let the user draw a rectangular ROI with the mouse; count and display only objects detected inside it, logging events to CSV.

GENERATED FILES

 roi_detection_log.csv
 snapshots/ snapshot_001.jpg ...

Smart surveillance

Factory safety

Retail counting

Smart parking

Lab Exercise 7.1 Interactive ROI Detection.py

EXPECTED OUTPUT

```
FPS : 32.4  
Confidence : 0.50  
In ROI : 5 objects  
  
Class statistics  
Person : 3  
Bottle : 1  
Chair : 1
```


Count a class, act on a threshold

Count specific object classes in real time and trigger alerts when a configurable threshold is crossed.

① People entering a building

② Vehicles in a parking lot

③ Defective products on a conveyor

IN PRODUCTION, THE ALERT BECOMES...

→ An **MQTT message** to a broker

→ An **email notification**

→ A **PLC relay** activation

Lab Exercise 7.3: People Counter with Alert System

Occupancy monitoring

CONFIGURATION

```
ALERT_THRESHOLD = 3
TARGET_CLASS = 0 # person
```

Count people each frame; when the count exceeds the threshold, flip the status bar to red and flash a border.

Retail compliance

Warehouse safety zones

Clean-room limits

People: 2 / 3

● Normal

People: 5 / 3

▲ OVER CAPACITY

status bar switches state at the threshold

Lab Exercise 7.3: People Counter with Alert System

SECTION

08

Object Tracking & Video Analytics

Persistent identity across frames — the foundation of counting, dwell time, and movement analytics.

Adding memory to vision

DETECTION

What is in this frame?

Identifies objects in each frame independently — every frame treated in isolation, with **no concept of identity across time**.

TRACKING

Which object is which, over time?

Assigns **persistent IDs** and maintains them across consecutive frames.

Unique visitors

Dwell time

Speed & trajectory

Tracking in one method call

```
# swap model() for model.track(frame, persist=True) # BoT-SORT / ByteTrack
```

BoT-SORT keeps identities stable — even through brief occlusions — by combining three signals:

Motion

A Kalman filter predicts where each object goes next.

Appearance

Re-ID features match objects by how they look.

Association

IoU overlap links detections to existing tracks.

Trails & unique counts

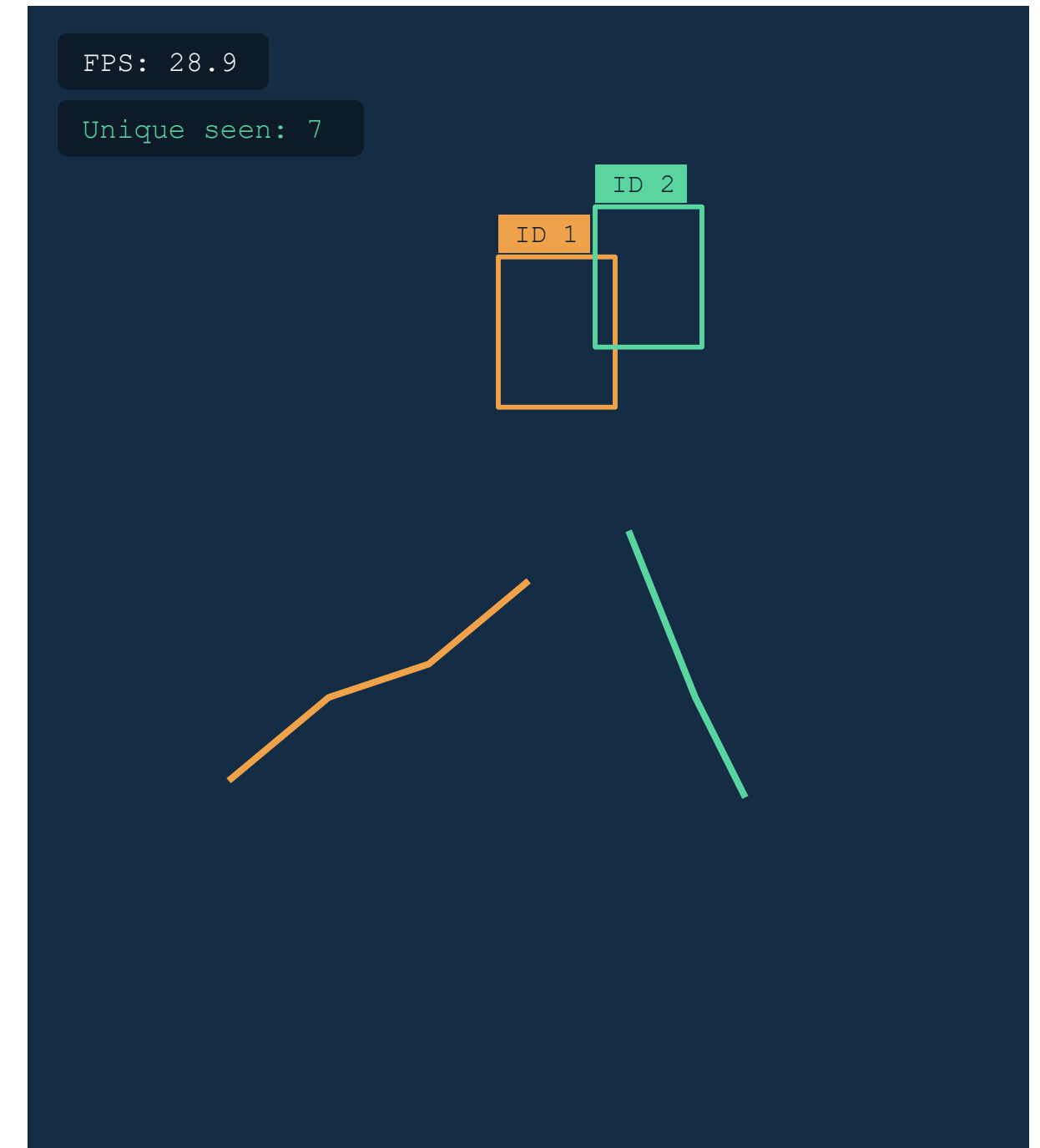
OBJECTIVE

Track objects across frames, draw a fading movement trail for each, and count the total unique objects seen.

KEY IDEAS

- Store recent centers per ID in a `track_history`
- Draw lines between them with a fading color
- Add each ID to a `unique_ids` set

EXPECTED OUTPUT



Count what crosses the line

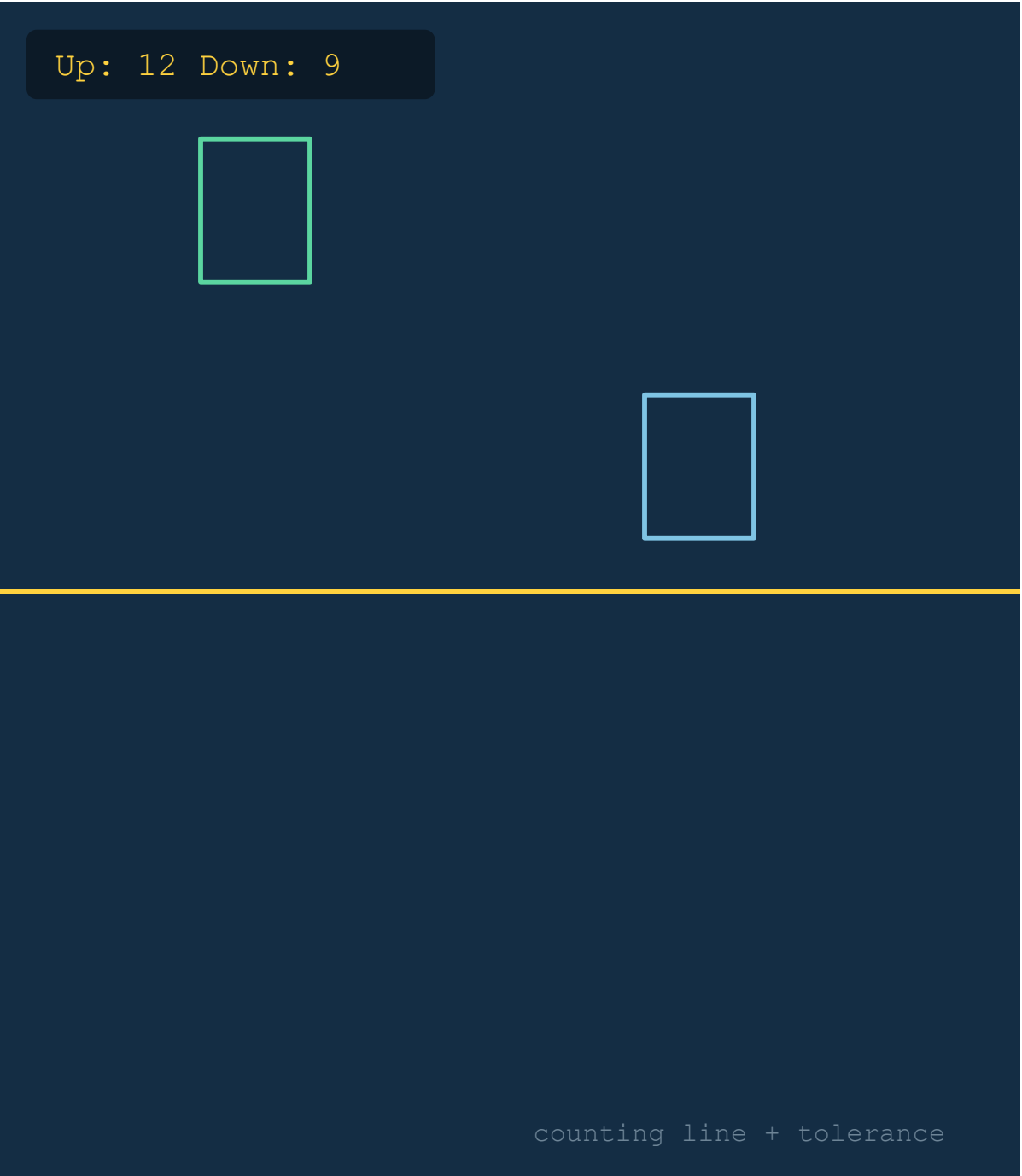
A virtual line counts objects as they cross — in both directions. Compare each tracked object's previous and current position against the line, within a tolerance zone, counting each ID once.

USED FOR

- Foot traffic
- Highway vehicles
- Conveyor items
- Retail flow

EXPECTED OUTPUT

Up: 12 Down: 9



counting line + tolerance

From primitives to platforms

Commercial video analytics combines these tracking & counting primitives into five capabilities:

01 **Heatmap generation**

High-traffic zones over time

03 **Speed & trajectory**

Velocity from position history

05 **Multi-camera handoff**

Re-ID across camera views

02 **Dwell-time analysis**

How long objects stay in a zone

04 **Anomaly detection**

Loitering, wrong-way, abandoned objects

Key takeaways

01 Edge inference wins where it counts — **latency, privacy, bandwidth, and cost.**

02 The practical stack is **YOLO + ONNX Runtime** — fast, portable, and small.

03 Optimization makes models fit — **start with FP16 quantization**, then measure.

04 Tracking turns detection into **analytics** — identity, counting, and flow.

05 All of it runs **locally on a Windows workstation** — no cloud required.

NEXT STEPS



Build it, benchmark it, ship it.

Document your edge projects on GitHub with benchmarks and demo videos — a working portfolio is one of the strongest hiring signals in the field.

REFERENCE DOCS

docs.ultralytics.com

onnxruntime.ai

opencv.org

pytorch.org